

LECTURE-24

Tuesday.

12/5/20.

⇒ Hash Page Table: Decrease the no. of pages in page table to reduce size using hashing.

→ same as Hash Table

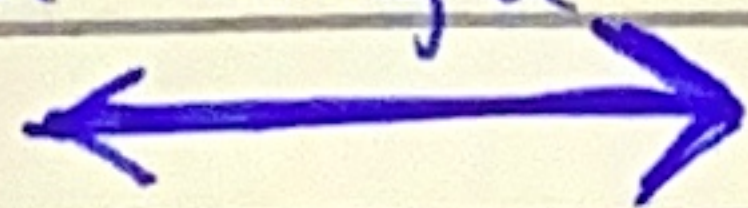
Method: Take modulus (%) of number and answer will give us the index of Hash table. That index will have page no., frame no. and

⇒ Inverted Page Table: Suppose we've 3 page tables with 100 entries each (total=300). BUT we've just 100 frames in RAM. Therefore when we'll allocate 100 frames ^{out} of 300 pages; and 200 pages would be not having any frame no. in their columns of frame no. Those 200 pages will be ~~being~~ contributing to wasting the memory.

Solution: make inverted Page table in which we'll just have a single table which will have frames as index means 100 index and so frame j's process key j's entry ko allocated hoga; uskey agay we'll mention with the ID of page table as we've 45th entry in all 3 pages.

⇒ Hierical Page Table.

EK process ka page table he itna bara key vo single page pr pura nai aa rha ∴ we'll use a second degree page table that POINTS to first page table. Second degree can have more pointing tables until we've a table that can get fixed in a single page.



Dehlay page table main reference frame likh rahay thay; but in inverted, we'll do key frames key samnay ham page no. likh dein gay jinko ye frame allocate hony. ∴ 200 pages key column main NULL nai ayega.

Q: Process Size / Logical address = 2GB
 Page Size = 1KB
 RAM = 512MB | Page Entry Size = 4 Bytes.

⇒ No. of Bytes to make a page table:

Index	Frame No.	C	D	T
1				
2				
...				
256				

★ Most space taken by frame No.
 and we're 3 bits extra
 for Control Bits (C, D, T).
 → one row = 4 Bytes (in 9k)

Q: Design an appropriate page table structure and calculate no. of levels required with no. of bit required at each level of hierarchical page table in order to fit it in exactly one frame of size given in ques. → 1KB as page size = 1KB.

• Logical address Bits = 2GB = 2×2^{30} Bytes = 2^{31} → 31 Bits

• Physical address Bits = 512MB = 2^{29} Bytes → 29 Bits

• Offset Bits (from page size) = 1KB = 2^{10} Bytes → 10 Bits

★ As logical address consist of "No. of Pages" + "Offsets"

$$\therefore 31 = p + 10$$

(p) Bits for no. of pages = 21

• Frames = RAM / Page Size

$$= 2^{29} / 2^{10}$$

$$= 2^{19} \text{ frames}$$

~~having size of 1KB & each frame having 256 entries of 4 Bytes each.~~
 Ram has 19 frames with each frame

Ram has 219 frames with each frame having size of 1KB & each frame has (1KB/4Bytes) 256 entries of 4 Byte each

• How many entries fit in one frame.

$$\text{Entries per Page Table} = \text{Page Size} / \text{Entry Size}$$

• Entries per P.T = $1\text{KB} / 4\text{ Bytes}$
 $= 2^{10} / 2^2$
 $= (2)^8$
 $= 256\text{ entries}$

• Total Pages in our process = 2^{21} (calculated previously)

∴ PT Size = $2^{21} \times 4\text{ Bytes}$
 $= 8\text{ MB}$

Condition for Page table to fit in 1 frame of 1KB size is:

~~8 MB~~ Page Table Size $\leq$ Frame/Page Size

$8\text{ MB} \not\leq 1\text{KB}$

this table.

∴ we'll keep making a new table which will point ↑

~~Verticalize~~

→ As one table can only hold $(2)^8$ entries but we've 2^{21} in our process;
 ∴ we'll make a level 2 table.

Level 2
 • $2^{21} / 2^8 = 2^{13}$
 PT Size = $2^{13} \times 4\text{ Bytes}$
 $= 2^{15}\text{ Bytes}$
 $= 32\text{ KB}$

★ As $32 \not\leq 1\text{KB}$, move to next Page Table.

Level 3.
 • $2^{13} / 2^8 = 2^5$
 PT Size = $2^5 \times 4\text{ Bytes}$
 $= 2^5 \times 2^2$
 $= 2^7 = 128\text{ Bytes}$
 ★ $128\text{ Bytes} \leq 1\text{KB}$ ✓
 ∴ we'll stop at level 3.

Final Split of Logical Address:

5	8	8	10
---	---	---	----

$(2)^5$
 entries in level 3
 page table

$(2)^8$ entries
 in level 2
 page table

$(2)^8$ entries
 in level 1
 page table
 offset/page size
 $= 1\text{KB}$

Both will be equal
 always.

⇒ Virtualization: Ek he powerful hardware main Hypervisor install krein which lets us use virtual machines to install other OS.

- Problem: Memory boht waste hoti & inefficient when managing CPU.

⇒ Container: Software kay sath baki dependencies aur environments install ho kr aati hain as a package jis ki wajah we don't need to create virtual machines as stand alone package for specific software is available jo kisi bhi OS par chal jaye ga mostly as package main environment sath aati hai. e.g: Managing software is Docker.

Problem: • They have some security issues as we don't have a specific environment like in VM.

- We are also very much dependent upon host OS.

⇒ Hybrid: VM + Container.
Removes security concerns.